Assignment 3: Revenue of Intercollegiate Athletic Programs in the USA

FEM11149 - Introduction to Data Science

Olivia Windisch

October, 2026

1. Introduction

Intercollegiate athletic programs are an important part of higher education in the United States, and running such a program requires substantial financial resources. However, not every program generates the same amount of revenue. While some larger athletic programs may earn over $100 million USD per year, many programs at smaller colleges have less than $1 million USD available for their teams. For college athletic directors, it is therefore useful to know what revenue can be expected for a program with certain characteristics. This may aid them with planning their budget and comparing their program to similar institutions. Colleges and universities that participate in federal student aid programs have to report information about their athletic programs to the US Department of Education (USDE), for example about the number of college athletes, as well as the coaches and their respective salaries. Many of these characteristics are closely related, since larger programs usually have more athletes and more coaches at the same time. This makes it relevant to investigate whether this information can be used to estimate the revenue of a program, and whether a few general features of a program already contain as much information as all characteristics together.

Using the reported information from 1,000 colleges and universities, the aim of this report is therefore to answer two research questions: To begin with, *to what extent can the revenue of an intercollegiate athletic program be predicted from a few underlying program characteristics, compared with using all characteristics separately?* Secondly, *which characteristics, either on their own or combined into these underlying features, are most closely related to revenue?*

2. Data

The data come from the Equity in Athletics Disclosure Act (EADA) survey of the USDE. Every college or university that participates in federal student aid programs and has at least one intercollegiate athletic program annually reports information about their programs (USDE, 2026). The dataset used for this report contains 1,000 institutions. Two variables are categorical: the classification of the athletic association and division (19 levels, e.g. NCAA Division I-FBS) and the sector of the institution (5 levels, e.g. private nonprofit). The other 28 predictor variables describe undergraduate enrollment, the number of athletes and teams, the participants per sport, the number and average salaries of head and assistant coaches, and the shares of full-time and female head coaches, mostly separately for men and women. The number of athletes counts each athlete once, while the participants per sport count an athlete once for every team (USDE, 2026). The unit ID identifies the institution and is not used as a predictor. The variable of interest is the total revenue of the athletic program, which includes for example ticket sales, media rights, donations and support from the institution. Since revenue has to cover the expenses of the program, a deficit paid by the institution also counts as revenue (USDE, 2025). As revenue is very right-skewed, the logarithm of revenue, which is much more symmetric (Appendix, Figure 1), is used as the dependent variable.

To evaluate the predictions on data that are not used for estimation, the 1,000 institutions are randomly split into a training set of 800 institutions (80%) and a test set of 200 institutions (20%).

Missing values only occur in the coach variables. Institutions without men's or women's teams report no coaches for those teams, so the number of coaches and their salaries are set to zero. For 10 institutions, assistant coaches are reported without a salary (16 missing salary values in total). As volunteer coaches are left out of the average salary (USDE, 2026), these may be programs with only volunteers. However, since this cannot be verified, the missing salaries are replaced by the median of the training data. The coach counts, the other counts and the salaries are also right-skewed, so they are transformed with $\log(1+x)$, which keeps zeros defined and reduces the influence of very large programs.

A second dataset contains 10 institutions for which the revenue is predicted. Their unit IDs are replaced by the numbers 1 to 10.

3. Methods

In order to reduce the number of predictors, *Principal Component Analysis (PCA)* is used. PCA is an unsupervised technique that tries to describe variables in a dataset as well as possible by a few new variables: the principal components. This is useful when many variables are strongly correlated, because they then measure more or less the same thing. The principal components are uncorrelated with each other and each component is a weighted sum of the original variables.

Let $\mathbf{X}$ be the $n \times p$ matrix of predictors $X_1, \dots, X_p$, with n observations and p predictors. Each principal component is a linear combination of the predictors, $Z_s = \gamma_{1s}X_1 + \gamma_{2s}X_2 + \cdots + \gamma_{ps}X_p = \gamma_s^\top \mathbf{X}$, where $\gamma_s = (\gamma_{1s}, \dots, \gamma_{ps})^\top$ is the loading vector with the weights of component s. The first component has the largest possible variance:

$$\max_{\gamma_1 : \|\gamma_1\|=1} \mathrm{Var}(\gamma_1^\top \mathbf{X}) = \max_{\gamma_1 : \|\gamma_1\|=1} \gamma_1^\top \Sigma \gamma_1,$$

where $\Sigma = \mathrm{Var}(\mathbf{X})$ is the covariance matrix of the predictors. The restriction $\|\gamma_1\| = 1$ prevents an infinitely large variance. The solution is the first eigenvector of Σ and $\mathrm{Var}(Z_1)$ is the first eigenvalue λ_1. Each next component again has maximum variance, but is uncorrelated with the previous components. This procedure gives eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$, where $\lambda_s = \mathrm{Var}(Z_s)$ measures the importance of component s. Since predictors are often measured in different units and PCA is sensitive to scale, PCA is done on the correlation matrix. This standardizes the values. The eigenvalues then add up to the number of variables, $\sum_{s=1}^{p} \lambda_s = p$, and the proportion of variance accounted for by component s is $\mathrm{VAF}_s = \lambda_s / p$.

Four different methods are then used to select a meaningful number of dimensions. To begin with, *cumulative VAF* is used to decide an initial number of components. It should be at least 80%, but this can often lead to many components. Secondly, *Kaiser's rule* is applied. This method keeps all components with $\lambda_s > 1$. If a component equal to one standardized variable X_j was chosen, its eigenvalue would be exactly 1. A component with $\lambda_s > 1$ explains more than one original variable. Third, a *scree plot* is utilized. The eigenvalue λ_s is plotted against the dimension number s. Components up to the visible "elbow" in the plot are kept (not including the elbow itself). Lastly, a *permutation test* compares each observed eigenvalue λ_s to the eigenvalues of $N_{\mathrm{perm}} = 1000$ datasets in which the values within each column are randomly permuted. This removes the correlations between the predictors. For each permuted dataset, the PCA eigenvalues are stored and the $\alpha/2$ and $1 - \alpha/2$ percentiles are computed (with $\alpha = 0.05$). If λ_s lies above this interval, dimension s is significantly different from random data and is kept.

To check how stable the eigenvalues and VAF are, a *bootstrap* procedure is applied. In total, $B = 1000$ bootstrap datasets of size $n \times p$ are drawn by sampling rows of $\mathbf{X}$ with replacement. The 2.5 and 97.5 percentiles of the eigenvalues give a 95% confidence interval. An eigenvalue is significantly larger than 1 if the whole interval lies above 1, and a narrow interval for the cumulative VAF indicates a stable result.

To interpret the components, the component loadings are used. These are the correlations between the original variables and the component scores. For standardized variables, they are $\rho_{js} = \mathrm{cor}(X_j, Z_s) = \gamma_{js}\lambda_s^{1/2}$. These correlations lie between -1 and $+1$. If a component has high correlations with a group of related

variables, it can be given a name. In addition, a biplot is made. It shows the observations as points and the variables as arrows for the first two components. Finally, the fit per variable in r dimensions, $\sum_{s=1}^{r} \rho_{js}^2$, can be interpreted as the proportion of the variance of variable X_j that is accounted for by the first r components. If all p components are used, the fit is 1. It can be said that a value close to 1 means that the variable is well represented.

In the principal component regression (PCR), the dependent variable is regressed with Ordinary Least Squares (OLS) on the first r component scores and dummy variables for the categorical predictors:

$$y_i = \beta_0 + \sum_{s=1}^{r} \theta_s z_{is} + \sum_{l=1}^{L} \delta_l d_{il} + \varepsilon_i,$$

where y_i is the dependent variable for observation i, β_0 the intercept, z_{is} the score of observation i on component s with coefficient θ_s, d_{il} the l-th of L dummy variables with coefficient δ_l, and ε_i the error term. A component has a significant relationship with the dependent variable at the 5% level if $|t| > 1.96$, where $t = \hat{\theta}_s / \mathrm{SE}(\hat{\theta}_s)$ is the estimate divided by its standard error. The p-value can also be used. It is the probability of finding a t-value at least this large if the null hypothesis (component s has no relation with the dependent variable) is true. A component has a significant relationship with the dependent variable at the 5% level if $p < 0.05$. A higher R^2 means that more variance of the dependent variable is explained by the model.

In addition to the PCR, a *ridge regression* is applied. It is a type of multiple regression that penalizes the size of the coefficients. This stabilizes the estimates when there are many or strongly correlated predictors. Ridge regression minimizes

$$L(\beta_1, \ldots, \beta_q) = \underbrace{\sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{q} x_{ij} \beta_j \right)^2}_{\text{regression term}} + \underbrace{\lambda \sum_{j=1}^{q} \beta_j^2}_{\text{penalty term}},$$

where q is the number of predictors including the dummy variables, x_{ij} the value of predictor j for observation i, β_j the regression weight of predictor j, and $\lambda > 0$ the penalty parameter. Here y_i is measured in deviation from its mean, so that the intercept is not penalized. This λ is not the same as the eigenvalues λ_s in the PCA. Ridge regression shrinks the β_j towards zero and λ controls the strength of this shrinkage. As $\lambda \to 0$, the ridge estimates become the OLS estimates, and as $\lambda \to \infty$, they go towards zero. Since all weights are penalized equally, the numerical predictors are standardized to z-scores first. Ridge regression is preferred over LASSO since LASSO can set coefficients to zero and tends to keep only one of several strongly correlated predictors. Ridge regression keeps all of them, even if they go towards zero.

A good value of λ is chosen with K-fold cross-validation with $K = 10$. Each fold is left out once and the model is estimated on the other $K - 1$ folds. It is then used to predict the n_k observations in the left-out fold k. The error is

$$\mathrm{MSE}_k = \frac{1}{n_k} \sum_{i \in \text{fold } k} (y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE}_{\mathrm{CV}} = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \mathrm{MSE}_k},$$

where $\hat{y}_i$ is the out-of-sample prediction for observation i. A lower value is seen as better. Two values are considered: $\lambda_{\min}$, the value with the lowest $\mathrm{RMSE}_{\mathrm{CV}}$, and λ_{1SE}, the largest λ whose error is within one standard error of this minimum. The 1-SE rule gives a simpler model with more shrinkage.

Both models are estimated on the training set and compared on the test set with $\mathrm{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2}$ and $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$, where m is the number of test observations and $\bar{y}$ their mean. A lower RMSE and a higher R^2 mean better predictions. When the dependent variable is a logarithm, an error on the log scale can be read as a relative error (in %). To get predictions on the original scale, the predictions are transformed back with $\exp(\hat{y}_i)$. On the original scale, the RMSE is mostly determined by the largest observations.

4. Results

The first principal component has an eigenvalue of 13.80 and explains 49.3% of the variance of the 28 predictors (Appendix, Table 1). The four selection methods do not all agree with each other. While the 80% VAF method suggests eight components, Kaiser's rule picks six, the scree plot two, and the permutation test three. The bootstrapped Kaiser's rule chooses five components. Since the permutation test is the only method that compares the eigenvalues to those of random data, the final selected components are the three from the permutation test. Together, the components explain 64.4% of the variance. The narrow bootstrap interval [63.1%, 66.5%] shows that this result is stable.

The loadings (Appendix, Table 2) and the biplot (Appendix, Figure 2) show what the components mean. The first component correlates positively with all variables. The correlation is strongest with the number of athletes, teams and coaches. It can be said that it measures *program size*. The second component is high for programs at large institutions with high coach salaries and many full-time coaches, and low for programs with many teams and coaches but low(er) salaries. This component can be named *Quality vs Quantity*. The third component separates individual sports, such as track and field, from the team sports basketball, baseball and softball. *Individual vs Team Sports* describes this component well. It is relevant to note that football (a team sport) participation has almost no correlation (-0.04) with this component. The effect of football, the main revenue-driver at FBS institutions, is mostly captured in the first component. The PCR also picks up on its effect through the division dummy variables.

In the PCR, the components and dummies explain 94.5% of the variance of log revenue in the training data. *Program size* and *Quality vs Quantity* are statistically significant at the 0.1% level ($p < 0.001$). *Individual vs Team Sports* is not statistically significant ($p = 0.64$). The division of a program is very important for predicting revenue: On average, an NCAA Division I-FBS program earns about 11 times as much as a CCCAA program, ceteris paribus.

For ridge regression, cross-validation chose a very small penalty (λ_{min} = 5.0e-04), so the results of it are very similar to OLS. The largest ridge coefficients are the division dummy variables, followed by the salary of head coaches of men's teams.

On the test set, both models predict log revenue well (Appendix, Table 3). The PCR has the lowest RMSE on the log scale (0.425, $R^2 = 0.916$), so three components and the divisions contain almost the same information as all 28 characteristics. On the original scale, ridge regression with λ_{min} has the lowest RMSE (8.78 million USD). These errors mainly come from the largest programs. The seven FBS programs in the test set cause 71% of the squared error. The predicted revenues of the ten new institutions (Appendix, Table 4) range from 1.7 million USD for the NAIA Division I program to 115.2 (PCR) and 142.2 million USD (ridge) for the FBS program.

5. Conclusions and Discussion

This report investigated *to what extent the revenue of an intercollegiate athletic program could be predicted from a few underlying characteristics, compared with using all characteristics separately.* The results show that revenue can be predicted quite well on a log scale. Three principal components, together with the athletic division, predict log revenue on the test set with an R^2 of 0.916. On the log scale, the PCR predictions are slightly better than the ridge regression with all 28 characteristics. One can therefore say that a few underlying factors contain almost all of the relevant information. Nevertheless, a prediction is usually off by a factor of exp(0.425), so around 1.5, and the errors in USD are the largest for the biggest athletics programs. The models can be useful as a benchmark for athletic program directors, but should not be used for predicting the exact revenue of one program.

When it comes to *the characteristics most closely related to revenue*, the results showed that revenue is mostly related to the size of a program. It is also somewhat relevant how well its coaches are paid. Furthermore, the division matters a lot. FBS programs earn much more than similar programs in lower divisions.

This research has several limitations. Most importantly, the results only show correlation, not causation. The data are also reported by the institutions themselves and only describe a certain point in time. In

addition, some missing salaries were filled with the median for this analysis, but they may have actually been volunteer coaches with a salary of zero. Moreover, components were formed without looking at revenue, so football, which is a crucial sport for the largest programs, is not captured well by them. Finally, the models were compared on only one train/test data split. Differences between them may change with another split. Future research could for example use data over a span of several years and repeat the split several times to check whether the differences between the models are robust.

References

US Department of Education (USDE). (2026). *Equity in Athletics Data Analysis*. https://ope.ed.gov/athletics//#/, retrieved 29.10.2026

US Department of Education (USDE). (2025). *EADA survey: Total revenues screen instructions*. https://surveys.ope.ed.gov/athletics2025/wwwroot/documents/inst_1.pdf, retrieved 29.10.2026

Appendix

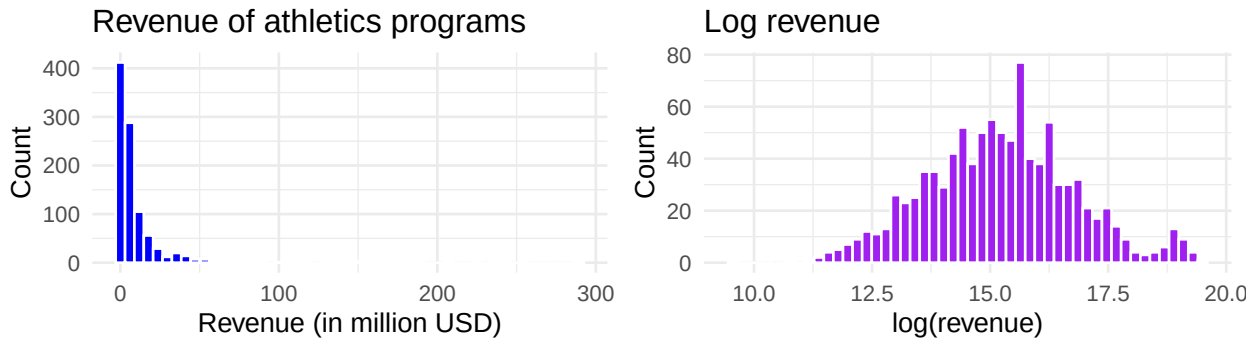


Figure 1: Distribution of revenue in million USD (left) and log revenue (right). Revenue is strongly right-skewed, log revenue is roughly symmetric.

Table 1: Eigenvalues, VAF (in %), upper bound of the permutation test and bootstrap confidence intervals of the eigenvalues for the first six components.

Component	Eigenvalue	VAF	Cumulative VAF	Permutation 97.5%	Bootstrap 95% CI
1	13.80	49.29	49.29	1.42	[13.28, 14.33]
2	2.74	9.77	59.07	1.35	[2.50, 3.09]
3	1.50	5.36	64.43	1.31	[1.39, 1.76]
4	1.23	4.40	68.83	1.27	[1.14, 1.49]
5	1.12	3.98	72.82	1.24	[1.04, 1.23]
6	1.02	3.65	76.47	1.21	[0.94, 1.10]

Table 2: Component loadings (correlations between variables and components C1 to C3) and fit per variable.

Variable	C1	C2	C3	Fit	Variable	C1	C2	C3	Fit
ugrad men	0.58	0.53	0.33	0.72	partic track women	0.78	0.15	0.24	0.70
ugrad women	0.53	0.57	0.36	0.73	partic volleyball women	0.63	-0.18	-0.20	0.47
athletes men	0.89	-0.15	-0.07	0.81	partic other men	0.75	-0.31	0.21	0.70
athletes women	0.94	-0.01	-0.02	0.88	partic other women	0.82	-0.08	0.27	0.75
teams men	0.86	-0.39	0.07	0.89	hc men	0.86	-0.38	0.05	0.89
teams women	0.92	-0.20	0.07	0.89	hc women	0.92	-0.20	0.05	0.90
partic football	0.59	0.09	-0.04	0.36	ac men	0.89	-0.03	0.07	0.79
partic bball men	0.48	-0.29	-0.48	0.55	ac women	0.87	0.03	0.14	0.79
partic bball women	0.60	-0.14	-0.39	0.53	hc salary men	0.64	0.52	-0.14	0.70
partic baseball	0.45	-0.18	-0.37	0.37	hc salary women	0.67	0.41	-0.24	0.68
partic softball	0.54	-0.10	-0.30	0.39	ac salary men	0.63	0.49	-0.26	0.70
partic soccer men	0.40	-0.51	0.14	0.44	ac salary women	0.63	0.46	-0.29	0.69
partic soccer women	0.70	-0.14	0.19	0.54	hc fulltime share	0.50	0.44	-0.21	0.49
partic track men	0.74	0.00	0.22	0.59	hc female share	0.13	0.25	0.11	0.09

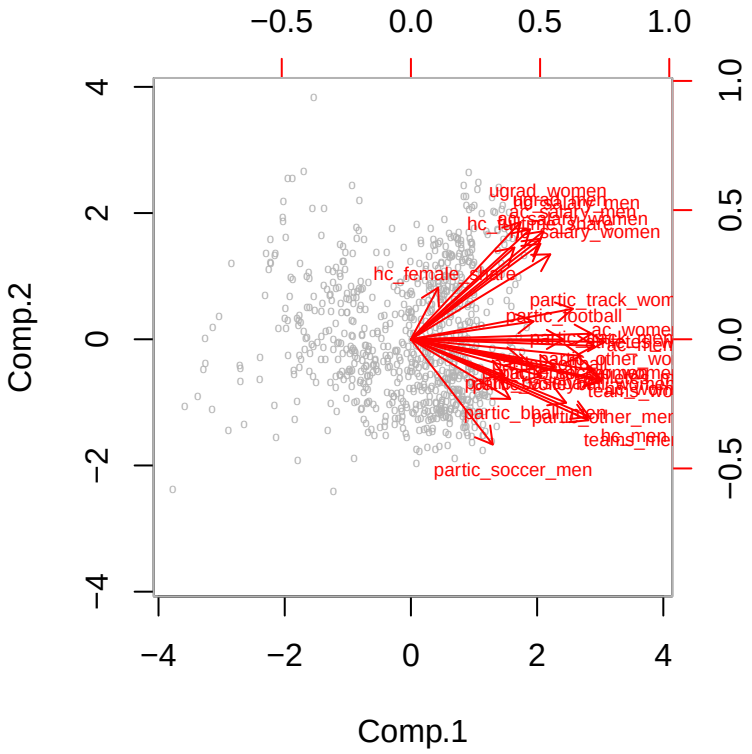


Figure 2: Biplot of components 1 and 2. Points are institutions, arrows are variables.

Table 3: Predictive performance on the test set.

Model	RMSE (log)	R^2 (log)	RMSE (million USD)
PCR	0.425	0.916	9.059
Ridge ($\lambda_{\min}$)	0.447	0.907	8.781
Ridge (λ_{1SE})	0.456	0.903	9.378

Table 4: Predicted revenue (in million USD) of the 10 institutions.

ID	Classification	PCR	Ridge
1	NCAA Division I-FBS	115.2	142.2
2	NCAA Division I-FCS	17.7	18.5
3	NCAA Division I without football	11.6	10.1
4	NCAA Division II with football	10.5	9.3
5	NCAA Division II without football	4.7	3.6
6	NCAA Division III with football	3.5	3.2
7	NCAA Division III without football	3.7	4.1
8	NAIA Division I	1.7	1.7
9	NJCAA Division I	2.2	2.7
10	CCCAA	4.4	4.7

Code

```
library(tidyverse); library(glmnet); library(boot); library(gridExtra); library(caret)

permtestPCA <- function(X, nTests = 100, alpha = 0.05, ...){ # permutation test function from lecture 5
  n <- nrow(X); m <- ncol(X); X <- scale(X)
  eigs.X <- prcomp(X)$sdev^2; eigs.Xperm <- matrix(0, m, nTests); Xperm <- X
  for (i in 1:nTests){
    for (j in 2:m) Xperm[, j] <- X[sort(runif(n), index.return = TRUE)$ix, j] # random permutation of the rows per column
    eigs.Xperm[, i] <- prcomp(Xperm)$sdev^2}
  perc.alpha <- t(apply(eigs.Xperm, 1, quantile, c(alpha/2, 1 - alpha/2)))
  plot(1:m, eigs.X, type = "b", col = "red", main = "Permutation test PCA", xlab = "Component", ylab = "Eigenvalue", ...)
  matlines(1:m, perc.alpha, type = "b", col = "blue", lty = 1)
  perc.alpha}
f2 <- function(x, d = 2) formatC(x, format = "f", digits = d)    # rounds numbers

athletics  <- read.csv("data_783996yw.csv"); prediction <- read.csv("predictions-2.csv")
dim(athletics); summary(athletics$revenue)                      # size and distribution of revenue
colSums(is.na(athletics))[colSums(is.na(athletics)) > 0]        # missing values only in the coach variables
table(no_men_teams = athletics$teams_men == 0, hc_missing = is.na(athletics$hc_men)) # no men's team -> no men's hc
min(athletics$ac_men, na.rm = TRUE)                             # ac never 0
sum(!is.na(athletics$ac_men) & is.na(athletics$ac_salary_men))  # acs but no salary (volunteers?)
table(athletics$classification); table(athletics$sector)        # some levels are very small
revenue <- ggplot(athletics, aes(revenue / 1e6)) + geom_histogram(bins = 50, fill = "blue", color = "white") +
  labs(title = "Revenue of athletics programs", x = "Revenue (in million USD)", y = "Count") + theme_minimal()
logrevenue <- ggplot(athletics, aes(log(revenue))) + geom_histogram(bins = 50, fill = "purple", color = "white") +
  labs(title = "Log revenue", x = "log(revenue)", y = "Count") + theme_minimal()
grid.arrange(revenue, logrevenue, ncol = 2)                     # very skewed, the log is more symmetric

count_vars  <- c("ugrad_men", "ugrad_women", "athletes_men", "athletes_women", "teams_men", "teams_women",
                 grep("^partic_", names(athletics), value = TRUE), "hc_men", "hc_women", "ac_men", "ac_women") # logged
salary_vars <- c("hc_salary_men", "hc_salary_women", "ac_salary_men", "ac_salary_women")                  # logged
share_vars  <- c("hc_fulltime_share", "hc_female_share")                                                 # 0 to 1, not logged
num_vars    <- c(count_vars, salary_vars, share_vars)                                                    # 28 numerical predictors
fixed_coaches <- function(x) {                                  # runs once for men's and once for women's variables
  for (s in c("men", "women")) {
    hc <- paste0("hc_", s); ac <- paste0("ac_", s); hs <- paste0("hc_salary_", s); as <- paste0("ac_salary_", s)
    no_hc <- is.na(x[[hc]]); no_ac <- is.na(x[[ac]])             # institutions without hcs / acs
    x[[hc]][no_hc] <- 0; x[[ac]][no_ac] <- 0                     # no coaches -> 0
    x[[hs]][no_hc & is.na(x[[hs]])] <- 0; x[[as]][no_ac & is.na(x[[as]])] <- 0 # coaches without salary stay na
  }
  x}
logged_vars <- function(x) { for (v in c(count_vars, salary_vars)) x[[v]] <- log1p(x[[v]]); x } # log(x + 1) as x can be 0
athletics_new  <- fixed_coaches(athletics); prediction_new <- fixed_coaches(prediction)
athletics_new$log_revenue <- log(athletics_new$revenue)         # dependent variable
athletics_new$classification <- factor(athletics_new$classification); athletics_new$sector <- factor(athletics_new$sector)
prediction_new$classification <- factor(prediction_new$classification, levels = levels(athletics_new$classification)) # same levels -> same dummies
prediction_new$sector <- factor(prediction_new$sector, levels = levels(athletics_new$sector))
athletics_new <- logged_vars(athletics_new); prediction_new <- logged_vars(prediction_new)

set.seed(123)
trainRowNumbers <- createDataPartition(athletics_new$log_revenue, p = 0.8, list = FALSE)
train_data <- athletics_new[trainRowNumbers, ]; test_data <- athletics_new[-trainRowNumbers, ]
train_medians <- sapply(train_data[num_vars], median, na.rm = TRUE) # training data only
impute <- function(x) { for (v in num_vars) x[[v]][is.na(x[[v]])] <- train_medians[v]; x }
train_data <- impute(train_data); test_data <- impute(test_data); prediction_new <- impute(prediction_new)
round(sort(sapply(train_data[num_vars], cor, y = train_data$log_revenue), decreasing = TRUE), 2) # correlations with log revenue

X_pca <- train_data[, num_vars]; X_pca_test <- test_data[, num_vars]; p <- ncol(X_pca)
fitpca <- princomp(X_pca, cor = TRUE, scores = TRUE)            # correlation matrix
eigenvalues <- fitpca$sdev^2; vaf <- eigenvalues / p; cum_vaf <- cumsum(vaf)
screeplot(fitpca, npcs = p, type = "lines", main = "Scree plot"); abline(h = 1, col = "red", lty = 2) # elbow and kaiser line
k_kaiser <- sum(eigenvalues > 1)                                # kaiser: eigenvalue > 1
k_vaf80  <- which(cum_vaf >= 0.80)[1]                           # 80% cumulative vaf
set.seed(123); perm_range <- permtestPCA(X_pca, nTests = 1000)  # permutation test
k_perm <- which(eigenvalues <= perm_range[, 2])[1] - 1          # keep pcs above the 97.5% interval bound
my_boot_pca <- function(x, ind) princomp(x[ind, ], cor = TRUE)$sdev^2 # pca on resampled rows -> eigenvalues
set.seed(123); eigs.boot <- boot(data = X_pca, statistic = my_boot_pca, R = 1000)$t
kaiser_boot <- data.frame(PC = 1:p, Eigenvalue = eigenvalues, Lower = apply(eigs.boot, 2, quantile, 0.025),
                          Upper = apply(eigs.boot, 2, quantile, 0.975))
k_kaiser_boot <- which(kaiser_boot$Lower <= 1)[1] - 1           # bootstrapped kaiser, whole confidence interval above 1
data.frame(Method = c("Kaiser", "80% VAF", "Permutation test", "Bootstrapped Kaiser"),
           PCs = c(k_kaiser, k_vaf80, k_perm, k_kaiser_boot))
n_pc <- k_perm                                                  # chosen: permutation test
vaf.boot   <- rowSums(eigs.boot[, 1:n_pc, drop = FALSE]) / p    # bootstrapped vaf of the chosen pcs
perc.alpha <- quantile(vaf.boot, c(0.025, 0.975)); round(100 * c(cum_vaf[n_pc], perc.alpha), 1)
corpca  <- cor(X_pca, fitpca$scores)[, 1:n_pc]                  # loadings
fit_var <- cbind(corpca, Fit = rowSums(corpca^2)); round(fit_var, 2) # fit per variable
biplot(fitpca, pc.biplot = TRUE, scale = 1, choices = 1:2, col = c("grey", "red"), asp = 1,
       cex = c(0.4, 0.6), xlabs = rep("o", nrow(X_pca)))      # points = institutions, arrows = variables

train_scores <- data.frame(fitpca$scores[, 1:n_pc], classification = train_data$classification,
                           sector = train_data$sector, log_revenue = train_data$log_revenue)
test_scores  <- data.frame(predict(fitpca, newdata = X_pca_test)[, 1:n_pc], # test projected on the training pca
                           classification = test_data$classification, sector = test_data$sector)
fit_pcr <- lm(log_revenue ~ ., data = train_scores); summary(fit_pcr)

form_x <- as.formula(paste("~", paste(c(num_vars, "classification", "sector"), collapse = " + ")))
y <- train_data$log_revenue
X <- model.matrix(form_x, train_data)[, -1]                     # dummies, intercept removed
X_test <- model.matrix(form_x, test_data)[, -1]; X_new <- model.matrix(form_x, prediction_new)[, -1]
x_mean <- colMeans(X[, num_vars]); x_sd <- apply(X[, num_vars], 2, sd) # z-scores with training means and sds
X[, num_vars] <- scale(X[, num_vars], x_mean, x_sd); X_test[, num_vars] <- scale(X_test[, num_vars], x_mean, x_sd)
X_new[, num_vars] <- scale(X_new[, num_vars], x_mean, x_sd)
grid <- 10^seq(-5, 2, length.out = 100)                         # grid of lambdas
plot(glmnet(X, y, alpha = 0, lambda = grid, standardize = FALSE), xvar = "lambda") # coefficient paths
set.seed(123); result.ridge.cv <- cv.glmnet(X, y, alpha = 0, lambda = grid, nfolds = 10, standardize = FALSE)
c(result.ridge.cv$lambda.min, result.ridge.cv$lambda.1se)       # lambda.min and 1se rule
result.ridge.min <- glmnet(X, y, alpha = 0, lambda = result.ridge.cv$lambda.min, standardize = FALSE)
result.ridge.1se <- glmnet(X, y, alpha = 0, lambda = result.ridge.cv$lambda.1se, standardize = FALSE)
round(cbind(result.ridge.min$beta, result.ridge.1se$beta), 3)   # coefficients per sd for numerical predictors
```

```
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
y_test <- test_data$log_revenue; rev_test <- test_data$revenue
pred_log <- list("PCR" = predict(fit_pcr, newdata = test_scores),
                 "Ridge lambda.min" = as.numeric(predict(result.ridge.min, newx = X_test)),
                 "Ridge lambda.1se" = as.numeric(predict(result.ridge.1se, newx = X_test)))
eval_tab <- do.call(rbind, lapply(names(pred_log), function(m) { pl <- pred_log[[m]]
  data.frame(Model = m, RMSE_log = rmse(y_test, pl), R2_log = 1 - sum((y_test - pl)^2) / sum((y_test - mean(y_test))^2),
             RMSE_rev_M = rmse(rev_test, exp(pl)) / 1e6) }))   # rmse in million usd after exp()
eval_tab
err_by_class <- data.frame(classification = test_data$classification,   # where do the usd errors come from?
                           sq_err_rev = (rev_test - exp(pred_log[["Ridge lambda.min"]]))^2) %>%
  group_by(classification) %>% summarise(n = n(), share_of_squared_err = sum(sq_err_rev)) %>%
  mutate(share_of_squared_err = share_of_squared_err / sum(share_of_squared_err)) %>% arrange(desc(share_of_squared_err))
err_by_class                                                 # fbs schools cause most of the usd error

new_scores <- data.frame(predict(fitpca, newdata = prediction_new[, num_vars])[, 1:n_pc],
                         classification = prediction_new$classification, sector = prediction_new$sector)
pred_out <- data.frame(id = prediction_new$id, classification = prediction_new$classification,
                       rev_PCR_M = exp(predict(fit_pcr, newdata = new_scores)) / 1e6,          # back to usd with exp()
                       rev_Ridge_M = exp(as.numeric(predict(result.ridge.min, newx = X_new))) / 1e6)
pred_out
```